SMART CONTRACT
SECURITY AUDIT REPORT

MANUMIT CORE TOKEN (MMC)

ERC-20 Token Contract — Full Security Inspection

Audit Date	June 16, 2026
Contract	MANUMITCORETOKEN.sol
Token	MANUMIT CORE (MMC)
Solidity	^0.8.34
Framework	OpenZeppelin (ERC20 + Ownable)
Overall Risk	**LOW — No Critical/High Issues Found**

— CONFIDENTIAL —

1. Executive Summary

This report presents the findings of a comprehensive security audit conducted on the MANUMIT CORE TOKEN (MMC) smart contract. The contract is an ERC-20 token implementation with a time-locked token release mechanism deployed on an EVM-compatible blockchain.

The audit covered manual code review, logic analysis, access control evaluation, economic security, and code quality assessment. The contract is built upon the well-audited OpenZeppelin library (ERC20 and Ownable modules) which significantly reduces the attack surface.

No critical or high severity vulnerabilities were identified. Two informational and one low-severity observations are documented in this report. The contract is considered safe for deployment pending acknowledgment of the centralization risk associated with owner privileges.

Severity	Count	Status
Critical	0	None Found
High	0	None Found
Medium	0	None Found
Low	1	Owner Centralization Risk
Informational	2	Lock Date Hardcoding + Event Completeness

2. Contract Overview

2.1 Token Economics

Parameter	Value	Notes
Token Name	MANUMIT CORE	
Token Symbol	MMC	
Total Supply	6,000,000,000 MMC	6 Billion tokens
Unlocked at Deploy	4,500,000,000 MMC	75% of supply
Locked Supply	1,500,000,000 MMC	25% — held in contract
Decimals	18	ERC-20 default
Lock Release Date	1846713600 (Unix)	Approx. Feb 14, 2028

2.2 Key Mechanisms

- Constructor mints 4.5B tokens to initialOwner and 1.5B tokens to the contract itself.
- releaseLockedTokens() transfers locked supply to the current owner after the LOCK_DATE has passed.
- A boolean flag lockedTokensReleased prevents double-release.
- remainingLockTime() and lockedBalance() provide view functions for transparency.

3. Audit Methodology

The audit was conducted through a structured multi-phase approach:

3.1 Phase 1 — Static Analysis

- Review of Solidity compiler version and pragma settings.
- Identification of imported dependencies and their security history.
- Check for known dangerous patterns: reentrancy, integer overflow/underflow, unprotected selfdestruct.
- Assessment of visibility modifiers, access control annotations, and function exposure.

3.2 Phase 2 — Logic & Business Rule Verification

- Verification that supply constants are consistent and non-contradictory.
- Examination of the time-lock mechanism and its enforcement.
- Analysis of the token release flow for edge cases and double-spend potential.
- Check of event emissions for indexing completeness and front-end observability.

3.3 Phase 3 — Access Control & Privilege Review

- Mapping of all owner-gated functions and their blast radius.
- Assessment of renounceOwnership() risk given the token lock mechanism.
- Review of the Ownable inheritance pattern and initialOwner validation.

3.4 Phase 4 — Economic & Game-Theory Analysis

- Review of incentives for token holders vs. the owner.
- Evaluation of owner's ability to manipulate release timing or beneficiary.
- Assessment of trust assumptions baked into the contract design.

4. Security Checklist — Full Results

The following checklist covers all standard ERC-20 and smart contract security checks. Each item is marked with a status.

#	Check	Status	Notes
1	**ACCESS CONTROL**		
2	onlyOwner modifier applied to privileged functions	**PASS**	releaseLockedTokens() is correctly gated
3	Zero-address check on owner in constructor	**PASS**	require(initialOwner != address(0)) present
4	No unprotected self-destruct or delegatecall	**PASS**	Neither is present in this contract
5	Ownership transfer pattern is safe (OZ Ownable)	**PASS**	Standard transferOwnership() is inherited
6	No hardcoded privileged addresses (except deployer)	**PASS**	initialOwner is dynamic parameter
7	**REENTRANCY & STATE MANAGEMENT**		
8	Checks-Effects-Interactions pattern followed	**PASS**	State flag set before _transfer()
9	No external calls to untrusted contracts	**PASS**	Only internal ERC-20 _transfer used
10	Double-release protection implemented	**PASS**	lockedTokensReleased boolean flag prevents replay
11	No ETH handling (payable/receive/fallback)	**PASS**	Contract holds no Ether
12	**ARITHMETIC & OVERFLOW**		
13	Solidity ^0.8.x built-in overflow protection active	**PASS**	No manual SafeMath needed in 0.8.x
14	Supply constants sum correctly (4.5B + 1.5B = 6B)	**PASS**	UNLOCKED + LOCKED == TOTAL_SUPPLY verified
15	No underflow risk in remainingLockTime()	**PASS**	Block.timestamp check guards subtraction
16	No unchecked arithmetic blocks present	**PASS**	No unchecked{} blocks anywhere
17	**ERC-20 COMPLIANCE**		
18	Full ERC-20 interface implemented (via OZ)	**PASS**	Inherits all required functions
19	transfer() and transferFrom() work correctly	**PASS**	OZ standard implementation
20	approve() and allowance() correctly implemented	**PASS**	OZ standard implementation
21	Transfer events correctly emitted	**PASS**	OZ handles Transfer events internally
22	Approval events correctly emitted	**PASS**	OZ handles Approval events internally

23	**TIME-LOCK MECHANISM**		
24	Block.timestamp used for time check (acceptable)	**PASS**	Minor miner manipulation tolerable at this scale
25	Lock date is immutable constant	**PASS**	Cannot be changed post-deployment
26	Tokens are held in contract address (not owner)	**PASS**	Contract self-holds locked supply
27	Release is one-time only (flag pattern)	**PASS**	lockedTokensReleased prevents replay
28	Lock date value verified as future date	**PASS**	1846713600 = approx. Feb 14, 2028
29	**CODE QUALITY & BEST PRACTICES**		
30	SPDX license identifier present	**PASS**	MIT license declared
31	Pragmas are specific and not overly permissive	**PASS**	^0.8.34 is acceptable
32	Events emitted for all state-changing operations	**INFO**	See Finding F-002 — additional indexed fields recommended
33	No magic numbers — constants used throughout	**PASS**	All values defined as named constants
34	Require messages are descriptive	**PASS**	Error strings are present and clear
35	No dead code or unreachable branches	**PASS**	All code paths are reachable
36	Constructor validates all critical inputs	**PASS**	Zero-address check enforced

5. Detailed Findings

F-001 — Owner Centralization Risk [LOW]

Severity	**LOW**
Location	releaseLockedTokens() — receiver is owner()
Status	Acknowledged — By Design

Description

The locked tokens (1.5 billion MMC, 25% of total supply) are unconditionally released to the current owner() at the time of the releaseLockedTokens() call. Ownership can be transferred via the inherited transferOwnership() function at any time. This means that if ownership is transferred before the release date, the new owner receives the locked tokens — not the original deployer.

Impact

A malicious or compromised owner could transfer ownership just before calling releaseLockedTokens(), redirecting 1.5B tokens. Additionally, calling renounceOwnership() after release is harmless, but doing so while tokens are still locked would permanently lock them (the contract holds them but nobody can call release). This is a trust assumption, not a code bug.

Recommendation

- Consider setting the beneficiary address at construction time and storing it as an immutable variable, rather than using dynamic owner().
- Alternatively, document explicitly that ownership must not be transferred until after token release.
- If renounceOwnership() should be blocked while tokens are locked, override it with a custom guard.

F-002 — Hardcoded Lock Date [INFORMATIONAL]

Severity	**INFORMATIONAL**
Location	LOCK_DATE = 1846713600
Status	Informational — No Immediate Action Required

Description

The lock date is a hardcoded Unix timestamp constant (1846713600). While this is technically correct and immutable, it reduces human readability and makes the date non-obvious in source code or on block explorers. The timestamp resolves to approximately February 14, 2028.

Recommendation

- Add an NatSpec comment above the constant explaining the human-readable date.
- Consider also adding a getter function that returns a string representation for front-end UX, though this is optional.

F-003 — Event Lacks Indexed Fields for Efficient Filtering [INFORMATIONAL]

Severity	**INFORMATIONAL**
Location	LockedTokensReleased event
Status	Note: receiver is already indexed

Description

The LockedTokensReleased event already marks the receiver as indexed, which is good. However, the amount field is not indexed. While indexing uint256 fields is uncommon (they cannot be compared as range queries on most EVM indexers), the event itself fires only once, so this is purely informational.

Recommendation

- No action strictly required. The event design is acceptable.
- If off-chain filtering by amount is needed in future, the amount field can be indexed.

6. Positive Security Observations

The following secure practices were observed and are commended:

- Checks-Effects-Interactions (CEI) pattern followed: lockedTokensReleased = true is set before _transfer() is called, preventing any reentrancy-style exploitation.
- Supply integrity: UNLOCKED_SUPPLY + LOCKED_SUPPLY equals TOTAL_SUPPLY exactly (4.5B + 1.5B = 6B), verified both mathematically and through constant declarations.
- Zero-address guard in constructor: The require(initialOwner != address(0)) prevents accidental deployment to a burn address.
- Use of trusted OpenZeppelin libraries: Both ERC20 and Ownable are from the well-audited OpenZeppelin codebase, drastically reducing the likelihood of standard implementation bugs.
- Solidity 0.8.x used: Built-in overflow and underflow protection eliminates an entire class of arithmetic vulnerabilities.
- No ETH handling: The contract holds no Ether and has no payable functions, eliminating ETH drain attack vectors.
- Immutable lock date: The LOCK_DATE constant cannot be altered post-deployment, guaranteeing the vesting schedule integrity.
- One-time release flag: The boolean lockedTokensReleased pattern is a simple, gas-efficient, and correct way to prevent repeated token releases.

7. Recommendations Summary

#	Priority	Recommendation	Finding
R1	MEDIUM	Store release beneficiary as immutable at construction to remove owner() dependency	F-001
R2	LOW	Add NatSpec comment on LOCK_DATE explaining the human-readable date	F-002
R3	LOW	Override renounceOwnership() with a guard while locked supply is unreleased	F-001
R4	INFO	Document trust assumptions around ownership transfer in project documentation	F-001

8. Conclusion

The MANUMIT CORE TOKEN (MMC) smart contract demonstrates a clean, well-structured implementation of an ERC-20 token with a time-locked vesting mechanism. The contract correctly leverages battle-tested OpenZeppelin components and follows key security best practices including the Checks-Effects-Interactions pattern, zero-address validation, and protection against double-release.

No critical or high-severity vulnerabilities were identified in this audit. The single low-severity observation relates to the inherent centralization of token release control in the owner address — a common design choice that represents a trust assumption rather than a technical vulnerability.

The contract is considered suitable for mainnet deployment. The team is advised to review and act upon the recommendations in Section 7, particularly R1 (beneficiary immutability) if a higher trust model is desired for the token lock mechanism.

OVERALL VERDICT	RISK RATING
APPROVED FOR DEPLOYMENT	LOW — 1 Low, 2 Informational

9. Disclaimer

This audit report was prepared for informational purposes only. While every effort was made to identify security vulnerabilities, no audit can guarantee the complete absence of bugs or vulnerabilities. This report does not constitute financial or legal advice. The findings and recommendations are based on the code provided at the time of review. Any subsequent modifications to the contract must be independently re-audited.